

CertNova Security Vulnerability Remediation Report

Date: 2026-05-30

Prepared by: Security Engineering Review

Executive Summary

This report documents the vulnerabilities identified and remediated in CertNova, with explicit mapping to vulnerability names, risk, affected areas, and implemented fixes.

Scope

- Backend API: 'server/index.js', 'server/coursesApi.js', 'server/db.js'
- Frontend Auth/API Flow: 'client/src/pages/AdminLogin.jsx', 'client/src/lib/api.js', 'client/src/admin/AdminDashboard.jsx', 'client/src/App.jsx'

Detailed Vulnerability Fixes

1) Hardcoded Administrative Credentials in Client

- Vulnerability name: Hardcoded Credentials (CWE-798)
- Severity: Critical
- Risk:
 - Admin credentials were embedded in frontend code ('admin' / 'password').
 - Any user inspecting JS could obtain admin access.
- Evidence before fix:
 - 'client/src/pages/AdminLogin.jsx' validated static username/password directly in browser logic.
- Fix implemented:
 - Removed hardcoded credential checks from client.
 - Added server-side authentication endpoint 'POST /api/admin/login'.
 - Client now submits credentials to server; server returns signed admin token only on successful validation.
- Files changed:
 - 'client/src/pages/AdminLogin.jsx'
 - 'server/index.js'

2) Missing Server-Side Authorization on Admin Endpoints

- Vulnerability name: Broken Access Control (OWASP A01:2021)
- Severity: Critical
- Risk:
 - Sensitive write operations were callable without verified admin identity.
 - Attackers could modify course/content/analytics data.
- Affected endpoints fixed:
 - 'POST /api/courses'
 - 'PUT /api/courses/:id'
 - 'PATCH /api/courses/:id/publish'
 - 'DELETE /api/courses/:id'
 - 'PUT /api/content/:id'
 - 'PUT /api/highlights/:id'
 - 'GET /api/analytics'

- 'POST /api/analytics/clear'
- 'POST /api/analytics/seed'
- Fix implemented:
 - Added admin token verification middleware ('requireAdmin') and request predicate ('isAdminRequest').
 - Applied checks to all sensitive admin routes.
- Files changed:
 - 'server/index.js'
 - 'server/coursesApi.js'

3) Unauthorized Access to Unpublished/All Course Data

- Vulnerability name: Broken Access Control / Insecure Direct Object Access Pattern
- Severity: Critical
- Risk:
 - '?all=1' parameter exposed all courses, including unpublished/admin data, with no server auth.
- Fix implemented:
 - For 'GET /api/courses?all=1' and 'GET /api/courses/:idOrSlug?all=1', server now requires valid admin token.
- Files changed:
 - 'server/coursesApi.js'
 - 'client/src/lib/api.js' (adds auth headers for admin reads)

4) Insecure Admin Session Trust (Client-Only Flag)

- Vulnerability name: Improper Authentication / Client-Side Trust of Security State
- Severity: High
- Risk:
 - Route guard trusted 'localStorage.adminAuth === 'true"' alone.
 - Local flag tampering could unlock admin UI flow.
- Fix implemented:
 - Client admin guard now requires both:
 - 'adminAuth === 'true''
 - presence of 'adminToken'
 - Admin logout removes both flag and token.
- Files changed:
 - 'client/src/App.jsx'
 - 'client/src/pages/AdminLogin.jsx'
 - 'client/src/admin/AdminDashboard.jsx'

5) Missing Auth Header Propagation for Admin API Calls

- Vulnerability name: Authorization Misconfiguration
- Severity: High
- Risk:
 - After backend auth hardening, client needed authenticated headers for admin actions.
 - Without this, flows would fail or encourage insecure bypasses.
- Fix implemented:
 - Added 'getAdminAuthHeaders()' helper.
 - Applied Authorization Bearer token on admin API operations and analytics fetch.

- Files changed:
 - 'client/src/lib/api.js'
 - 'client/src/admin/AdminDashboard.jsx'

6) Overly Permissive CORS Policy

- Vulnerability name: Security Misconfiguration (OWASP A05:2021)
- Severity: High
- Risk:
 - Open CORS may allow untrusted origins to issue browser-based requests.
- Fix implemented:
 - Replaced permissive CORS with allowlist-driven policy:
 - 'ALLOWED_ORIGINS' env var
 - default fallback: 'http://localhost:5173'
- Files changed:
 - 'server/index.js'

7) Verbose Internal Error Disclosure

- Vulnerability name: Information Exposure Through Error Messages (CWE-209)
- Severity: Medium
- Risk:
 - Returning raw 'err.message' can disclose DB/schema/system internals.
- Fix implemented:
 - Replaced outward-facing server errors with generic 'Internal server error' on sensitive paths.
- Files changed:
 - 'server/index.js'
 - 'server/coursesApi.js'

8) Insecure Default Database Connection Credentials

- Vulnerability name: Use of Hard-coded Credentials / Insecure Defaults
- Severity: Medium
- Risk:
 - Default DB URL included username/password in source fallback.
- Fix implemented:
 - Removed hardcoded credential fallback from DB connection string.
 - Server now expects 'DATABASE_URL' or local trust configuration.
- Files changed:
 - 'server/db.js'

9) Realistic Personal Data in Mock Analytics Seed

- Vulnerability name: Privacy Data Exposure / Data Minimization Failure
- Severity: Medium
- Risk:
 - Seed data contained realistic personal-looking emails/names.
- Fix implemented:
 - Replaced with neutral placeholder identities ('learnerX@example.com').
- Files changed:
 - 'server/db.js'

- 'server/index.js'

Security Controls Added

- HMAC-signed admin token generation and verification.
- Expiration support for admin tokens ('ADMIN_TOKEN_TTL_SECONDS').
- Constant-time credential comparison ('timingSafeEqual') to reduce timing leakage.
- Centralized admin auth helper propagation on frontend API layer.

Environment Variables Required

- 'ADMIN_USER'
- 'ADMIN_PASSWORD'
- 'ADMIN_TOKEN_SECRET'
- 'ALLOWED_ORIGINS'
- Optional: 'ADMIN_TOKEN_TTL_SECONDS'

Residual Risks (Not Yet Remediated)

1. Learner authentication is still localStorage-driven and not server-validated.
2. Certificate list endpoint by email ('/api/certificates?email=...') can still allow enumeration/privacy risk without user-bound auth.
3. No rate limiting/lockout on admin login endpoint.
4. No CSRF design review for future cookie-based auth migration.

Recommended Next Phase

1. Implement proper learner JWT/session auth and server-side authorization checks.
2. Lock certificate access to authenticated user identity (token subject), not email query alone.
3. Add rate limiting and login attempt controls on '/api/admin/login' and sensitive endpoints.
4. Add security headers ('helmet') and centralized audit logging.

Verification Summary

- Static syntax checks passed for modified server files via Node check.
- Admin flow updated end-to-end: login -> token storage -> protected route -> authenticated admin API calls.
- Sensitive admin endpoints now reject unauthorized requests with '401 Unauthorized'.